
Introdução ao R

Aula 00 — O básico para acompanhar o curso

Projeto Integrador III ■ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Por que esta aula

- Todo o curso usa **R**
- A Aula 01 já indexa textos
- Aqui: o mínimo para acompanhar

O que veremos

- Vetores e vetores **nomeados**
- Texto: `tolower`, `strsplit`
- Funções e `lapply`/`sapply`
- `table`, `factor` e matrizes

Meta

Ao final, você lê e escreve todo o código R que aparece na **Aula 01**.

- **R**: `cran.r-project.org` — a linguagem e o interpretador.
- **RStudio** (ou Positron): ambiente com editor, console e gráficos.
- Você digita no **console** e vê o resultado na hora — ideal para explorar dados.

Nesta disciplina usamos apenas **R base**: nada a instalar além do próprio R.

Em R, quase tudo é vetor. O operador de atribuição é `<-`.

```
v <- c(10, 20, 30, 40)  # c() = "combine": cria um vetor
v
sum(v)
[1] 10 20 30 40
[1] 100
```

inicia um comentário. Outras úteis: `length()`, `mean()`, `sort()`.

Vetores nomeados (muito usado no curso)

Cada posição pode ter um **nome** — é assim que guardamos nosso *corpus*.

```
notas <- c(ana = 8, bruno = 6, carla = 9)
notas
notas["bruno"]           # acesso pelo NOME, não pela posição
  ana bruno carla
    8     6     9
bruno
  6
```

Na Aula 01: docs <- c(d1 = "...", d2 = "...") e depois docs["d5"].

```
frase <- "recuperacao de informacao"
toupper(frase)
strsplit(frase, " ")      # quebra em pedacos -> devolve uma LISTA
[1] "RECUPERACAO DE INFORMACAO"
[[1]]
[1] "recuperacao" "de"          "informacao"
```

`tolower()` faz o inverso: deixa tudo minúsculo (usaremos muito).

`strsplit` devolve uma **lista**; `unlist` “achata” para um vetor simples.

```
| unlist(strsplit(frase, " "))
```

```
[1] "recuperacao" "de"           "informacao"
```

Por isso, na Aula 01, escrevemos `unlist(strsplit(texto, "\\s+"))`. O `"\\s+"` é uma **expressão regular**: “um ou mais espaços”.

Criando suas próprias funções

```
dobro <- function(x) {  
  x * 2          # a ultima expressao e o valor devolvido  
}  
dobro(7)  
[1] 14
```

Funções deixam o código reutilizável — `tokenizar()`, `busca_booleana()` etc.

Em vez de laços, o R usa a família apply. lapply devolve **lista**:

```
palavras <- list(a = c("x","y","z"), b = c("p","q"))  
lapply(palavras, length)
```

```
$a
```

```
[1] 3
```

```
$b
```

```
[1] 2
```

sapply: o mesmo, mas simplificado

sapply tenta devolver um **vetor** ou **matriz** em vez de lista.

```
| sapply(palavras, length)
```

```
a b
```

```
3 2
```

Regra prática: lapply quando quiser lista; sapply quando quiser vetor/matriz. Na Aula 01 usamos lapply para tokenizar e sapply para montar a matriz.

```
tokens <- c("de", "casa", "de", "rua")  
table(tokens)           # conta as ocorrencias de cada valor  
tokens  
casa  de  rua  
   1   2   1
```

É assim que obtemos a **frequência dos termos** do *corpus*.

factor: fixando as categorias

Sem factor, table conta apenas o que *aparece*. Com levels, fixamos **todas** as categorias — inclusive as de contagem **zero**.

```
vocab <- c("casa", "de", "rua", "praia")  
table(factor(tokens, levels = vocab))
```

```
casa    de    rua  praia  
    1     2     1     0
```

Isso garante que **todo documento** vire um vetor do mesmo tamanho: o do vocabulário.

```
m <- matrix(1:4, nrow = 2)
rownames(m) <- c("lin1", "lin2")
colnames(m) <- c("c1", "c2")
m
```

```
      c1 c2
lin1  1  3
lin2  2  4
```

Acesso: `m["lin1", ]` (linha), `m[, "c2"]` (coluna), `m["lin1","c2"]` (elemento).

- `nrow()`, `ncol()`, `dim()` — dimensões.
- `rowSums()`, `colSums()` — somas por linha/coluna.

```
peso <- c(2, 3)           # um peso por LINHA
m * peso                  # reciclagem: multiplica linha a linha

      c1 c2
lin1  2  6
lin2  6 12
```

Essa **reciclagem** é exatamente o que faz `tfidf <- tdm * idf` funcionar na Aula 01.

Comparações e o operador %in%

```
| c("casa", "aviao") %in% vocab      # cada elemento esta em vocab?  
[1]  TRUE FALSE
```

Com ! (negação), é como removemos *stopwords*: `t[!t %in% stopwords]`. Note também: `sort(x, decreasing = TRUE)` ordena do maior para o menor.

O que é

Uma **regex** é uma pequena linguagem para **descrever padrões** de texto — em vez de procurar uma palavra exata, você descreve a *forma* do que procura.

- Você **já viu uma**: "`\\s+`" significa “um ou mais espaços”.
- É universal: existe em R, Python, editores de texto e no terminal.
- Em R, as funções que aplicam regex são `grep`, `grep1`, `sub` e `gsub`.

Símbolo	Significado	Exemplo
<code>^</code>	início do texto	<code>"^ca"</code> — começa com “ca”
<code>\$</code>	fim do texto	<code>"a\$"</code> — termina em “a”
<code>[0-9]</code>	qualquer dígito	<code>"[0-9]"</code> — contém número
<code> </code>	ou (alternativa)	<code>"casa praia"</code>
<code>\\s</code>	espaço em branco	<code>"\\s+"</code> — um ou mais
<code>+</code>	uma ou mais repetições	<code>"a+"</code> — “a”, “aa”...

Só isso já resolve muita coisa. O resto se consulta quando precisar.

```
palavras <- c("casa", "cachorro", "praia", "cidade")
grep("^ca", palavras, value = TRUE)    # quais COMECAM com "ca"?
grepl("a$", palavras)                  # quais TERMINAM em "a"?

sub("a", "@", "banana")                 # troca a PRIMEIRA ocorrencia
gsub("a", "@", "banana")                 # troca TODAS

[1] "casa"      "cachorro"
[1] TRUE FALSE  TRUE FALSE
[1] "b@nana"
[1] "b@n@n@"
```

Guarde a diferença: `sub` troca uma vez; `gsub` troca todas (o “g” é de *global*).

Mapa: o que você já consegue ler

Na Aula 01 aparece	Você aprendeu aqui
<code>docs <- c(d1 = "...")</code>	vetor nomeado
<code>unlist(strsplit(x, "\\s+"))</code>	texto → tokens
<code>lapply(docs, tokenizar)</code>	aplicar função a vários
<code>table(factor(t, levels = vocab))</code>	contagem com categorias fixas
<code>sapply(...) → matriz</code>	montar a matriz termo-documento
<code>tdm * idf</code>	reciclagem

Tarefa principal

Instale o R e, para **cada** bloco de código desta aula:

- ① **Explicar**: escreva, com suas palavras, o que cada linha faz e por que ela existe.
- ② **Explorar**: rode o código, **altere** algo (um valor, um argumento, um nome) e registre *o que mudou no resultado*.
- ③ **Prever**: antes de rodar a versão alterada, escreva o que você *espera* que aconteça — e compare.

Entregue um documento curto com o código, sua explicação e as saídas observadas.

Explore e responda

- ① O que acontece se você trocar `sapply` por `lapply` no exemplo? E o contrário?
- ② Em `table(factor(tokens, levels = vocab))`, o que muda se você **remover** o `factor`? Por que isso importa para o nosso motor de busca?
- ③ Na reciclagem `m * peso`, o que acontece se `peso` tiver **3** elementos em vez de 2? Preveja antes de rodar.
- ④ Crie um vetor nomeado com **3 frases** suas, tokenize com `lapply` e conte os termos com `table`, ordenando do maior para o menor.

Exercício final: o faxineiro de manchetes

Você coletou manchetes de um jornal da Baixada Santista. Elas vêm **sujas**: com o nome do jornal grudado, espaços sobrando e formatação inconsistente.

```
manchetes <- c(
  "Porto de Santos bate recorde em julho - A Tribuna",
  "cubatao registra melhora na qualidade do ar - A Tribuna",
  "Guaruja tera nova linha de onibus em 2026 - A Tribuna",
  "Sao Vicente inaugura escola no Parque Bitaru - A Tribuna",
  "Santos e Guaruja discutem travessia de balsa - A Tribuna"
)
```

Seu trabalho: **limpar** e **investigar** esse material usando *regex*.

Missões (use grep, grepl, sub e gsub)

- 1 **Faxina:** remova o sufixo " - A Tribuna" de todas.
Dica: `sub(" - A Tribuna$", "", manchetes)` — por que o \$ importa aqui?
- 2 **Espaços:** troque dois ou mais espaços por um só.
Dica: `gsub("\\s{2,}", " ", x)`. Por que gsub e não sub?
- 3 **Ano:** quais manchetes citam um ano (4 dígitos)?
Dica: `"[0-9]{4}"`.
- 4 **Municípios:** quais citam **Guarujá ou Cubatão**, *ignorando* maiúsculas?
Dica: o | e o argumento `ignore.case = TRUE`.
- 5 **A pegadinha:** `grepl("Santos", manchetes)` acha 2 manchetes — mas uma fala do **Porto**, não da cidade. Como você distinguiria? É possível só com regex?

Por que este exercício importa

- Todo conjunto de textos real chega **sujo** — limpar é o primeiro passo de qualquer motor de busca.
- A missão 5 mostra o limite: regex acha **padrões**, mas não entende **sentido**. “Santos” pode ser a cidade, o porto ou o time.
- Resolver isso exige **semântica** — e é para lá que o curso caminha.

Entrega

O código de cada missão, o resultado obtido e um parágrafo respondendo à missão 5.

Próxima aula

- Recuperação de Informação
- Primeiro *corpus* e índice

Ajuda rápida

- `?funcao` abre a documentação
- `str(x)` mostra a estrutura de `x`
- `class(x)` diz o tipo de `x`

Guarde esta ideia

Você não precisa decorar o R. Precisa reconhecer **vetor**, **lista** e **matriz** — o resto se consulta.

Obrigado!

Dúvidas? Vamos ao R.
